



LINUX
SECURITY
SUMMIT

Triaging Kernel Out-of-Bounds Write Vulnerabilities

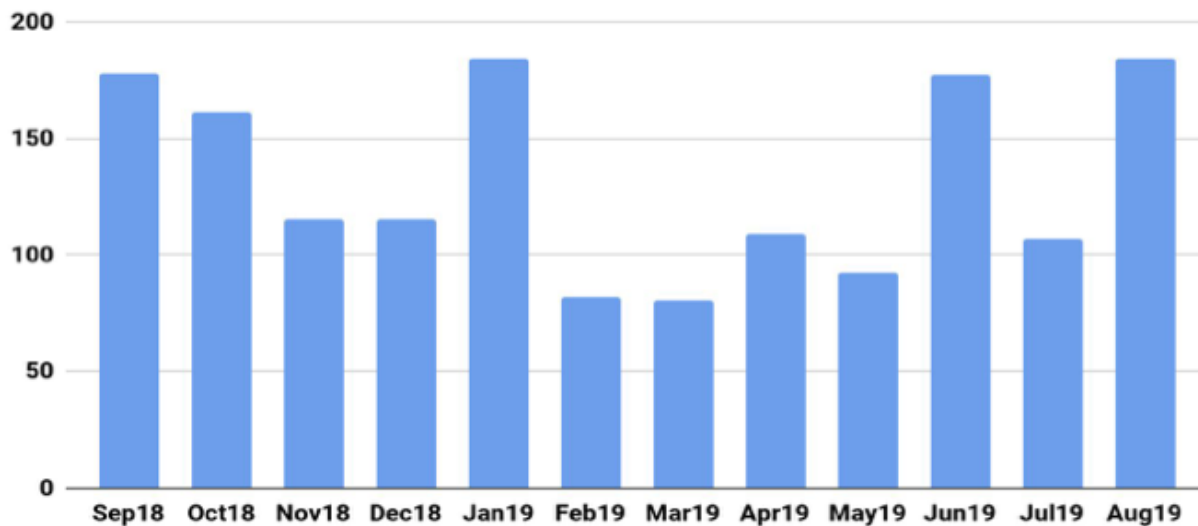
Weiteng Chen, Xiaochen Zou, Guoren Li, Zhiyun Qian
University of California, Riverside

@weitengchen



Introduction

- According to syzbot, Google's continuous kernel fuzzing platform, there were 1216 Linux kernel bugs discovered and fixed during a single year (from Aug 2017 to Sep 2018).



[1] syzbot: update and open problems at Linux Plumbers 2019.

[https://linuxplumbersconf.org/event/4/contributions/428/attachments/308/515/syzbot Plumbers 2019.pdf](https://linuxplumbersconf.org/event/4/contributions/428/attachments/308/515/syzbot_Plumbers_2019.pdf)

Introduction

- 1038 Open issues
 - ~75% were reported more than 100 days ago
- Average patch delay: 51 days (over 2705 fixed bugs)



Introduction

- Exploitability assessment based on vulnerability type
 - UAF/OOB Write bugs are likely exploitable
 - Still require more context to determine whether they are exploitable or not

Goal

- Given one PoC that triggers OOB Write, we want to ...
 - Facilitate the process of generating IP-hijacking primitives
 - Help analyst understand the root cause and security impact

Motivating Example

```
1. struct Type1 { ...; };
2. struct Type2 { Type1 sk; uint64_t option; ...; };
3. struct Type3 { int (*ptr)(); ...; };
4. struct Type4 { uint64_t state; Type3 *sk; ...; };
5. struct Type5 { atomic_t refcnt; ...; };
6. Type2 gsock = { ..., .option = 0x08080000000000, };
7. Type1 * vul = NULL; Type3 * tgt = NULL;
8. void sys_socket() //sizeof(Type1) == sizeof(Type3)
9. vul = kmalloc(sizeof(Type1))
```

← 1. Allocate the vulnerable object

CVE-2018-5703

Motivating Example

```
1. struct Type1 { ...; };
2. struct Type2 { Type1 sk; uint64_t option; ...; };
3. struct Type3 { int (*ptr)(); ...; };
4. struct Type4 { uint64_t state; Type3 *sk; ...; };
5. struct Type5 { atomic_t refcnt; ...; };
6. Type2 gsock = { ..., .option = 0x08080000000000, };
7. Type1 * vul = NULL; Type3 * tgt = NULL;
8. void sys_socket() //sizeof(Type1) == sizeof(Type3)
9.     vul = kmalloc(sizeof(Type1))
```

← 1. Allocate the vulnerable object

```
10. void sys_accept()
```

```
11.     vul = (Type2*)vul;           //type confusion
12.     vul->option = gsock.option; //Vulnerability Point
```

← 2. Overwrite the following object via the vulnerable object

CVE-2018-5703

Motivating Example

```
1. struct Type1 { ...; };
2. struct Type2 { Type1 sk; uint64_t option; ...; };
3. struct Type3 { int (*ptr)(); ...; };
4. struct Type4 { uint64_t state; Type3 *sk; ...; };
5. struct Type5 { atomic_t refcnt; ...; };
```

```
6. Type2 gsock = { ..., .option = 0x08080000000000, };
```

```
7. Type1 * vul = NULL; Type3 * tgt = NULL;
```

```
8. void sys_socket() //sizeof(Type1) == sizeof(Type3)
```

```
9. vul = kmalloc(sizeof(Type1))
```

```
10. void sys_accept()
```

```
11. vul = (Type2*)vul; //type confusion
```

```
12. vul->option = gsock.option; //Vulnerability Point
```

← 3. Overflow with a constant?

← 1. Allocate the vulnerable object

← 2. Overwrite the following object via the vulnerable object

CVE-2018-5703

Motivating Examples

```
1. struct Type1 { ...; };
2. struct Type2 { Type1 sk; uint64_t option; ...; };
3. struct Type3 { int (*ptr)(); ...; };
4. struct Type4 { uint64_t state; Type3 *sk; ...; };
5. struct Type5 { atomic_t refcnt; ...; };
```

```
6. Type2 gsock = { ..., .option = 0x0808000000000000, };
```

```
7. Type1 * vul = NULL; Type3 * tgt = NULL;
```

```
8. void sys_socket() //sizeof(Type1) == sizeof(Type3)
```

```
9. vul = kmalloc(sizeof(Type1))
```

← 1. Allocate the vulnerable object

```
10. void sys_accept()
```

```
11. vul = (Type2*)vul; //type confusion
```

```
12. vul->option = gsock.option; //Vulnerability Point
```

← 2. Overwrite the following object via the vulnerable object

```
13. void sys_setsockopt(val) //not invoked in given PoC
```

```
14. if (val == -1) return;
```

```
15. gsock.option = val;
```

← 3. Control the overflowed data

CVE-2018-5703

Challenge 1: Exploration

- The initial PoC does not manifest the complete capability the corresponding vulnerability has.
 - “SyzScope: Revealing High-Risk Security Impacts of Fuzzer-Exposed Bugs”: 147 out of 1173 seemingly low-risk bugs (eg, OOB read) have high impacts (eg, IP-hijacking).

Challenge 2: Modeling Capability

```
void example1(size)
    vul = kmalloc(size);
    vul[size] = '\0';
```

CVE-2016-6187

size and i are user controllable.

```
void example2(i)
    vul = (char*)kmalloc(sizeof(TYPE));
    //omit other OOB points on the path
    vul[i/8] |= 1<<(i&0x7); //set 1 bit
```

CVE-2017-7184

Challenge: Modeling Capability (Cont'd)

```
void example1(size)
    vul = kmalloc(size);
    vul[size] = '\0';
```

CVE-2016-6187

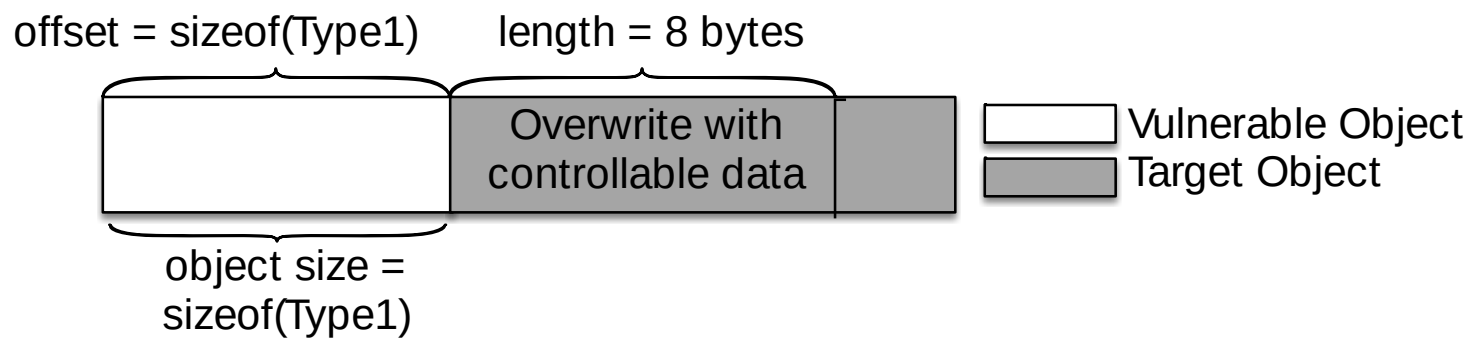
```
void example2(i)
    vul = (char*)kmalloc(sizeof(TYPE));
    //omit other OOB points on the path
    vul[i/8] |= 1<<(i&0x7); //set 1 bit
```

CVE-2017-7184

Different OOB vulnerability instances exhibit a wide range of capabilities, in terms of:

- 1) how far the write can reach**
- 2) how many bytes can be written**
- 3) and what value can be written.**

Challenge 3: Exploitability Evaluation



Vuln Object's size: $s = \text{sizeof}(\text{Type1})$

Offset: $o = \text{sizeof}(\text{Type1})$

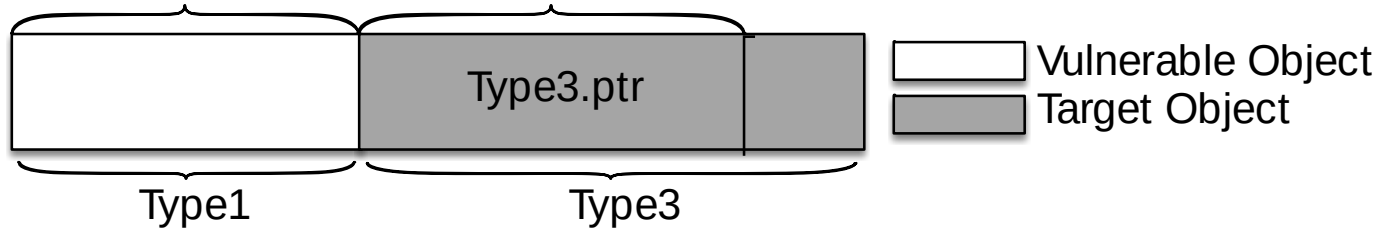
Length: $l = 8 \text{ bytes}$

Value: $v = 0 \sim 0\text{xffffffffffffffff}$

- The effect of the OOB write depends on the target object following the vulnerable object.

Challenge 3: Exploitability Evaluation

offset = sizeof(Type1) length = 8 bytes



Vuln Object's size: $s = \text{sizeof}(\text{Type1})$

Offset: $o = \text{sizeof}(\text{Type1})$

Length: $l = 8 \text{ bytes}$

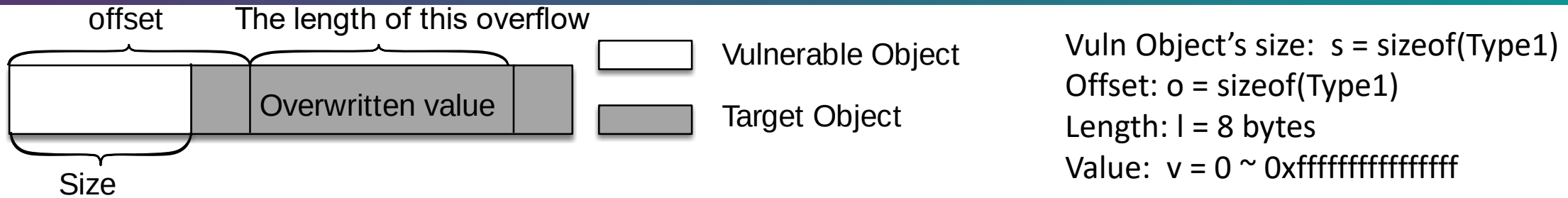
Value: $v = 0 \sim 0\text{xffffffffffffffff}$

```
1. struct Type1 { ...; };
2. struct Type2 { Type1 sk; uint64_t option; ...; };
3. struct Type3 { int (*ptr)(); ...; };
4. struct Type4 { uint64_t state; Type3 *sk; ...; };
5. struct Type5 { atomic_t refcnt; ...; };
6. Type2 gsock = { ..., .option = 0x08080000000000, };
7. Type1 * vul = NULL; Type3 * tgt = NULL;
8. void sys_socket() //sizeof(Type1) == sizeof(Type3)
9.     vul = kmalloc(sizeof(Type1))
... ..
16. void sys create tgt()
17.     tgt = kmalloc(sizeof(Type3));
18.     tgt->ptr = NULL; //init ptr
```

Our target object with a function
pointer at the beginning

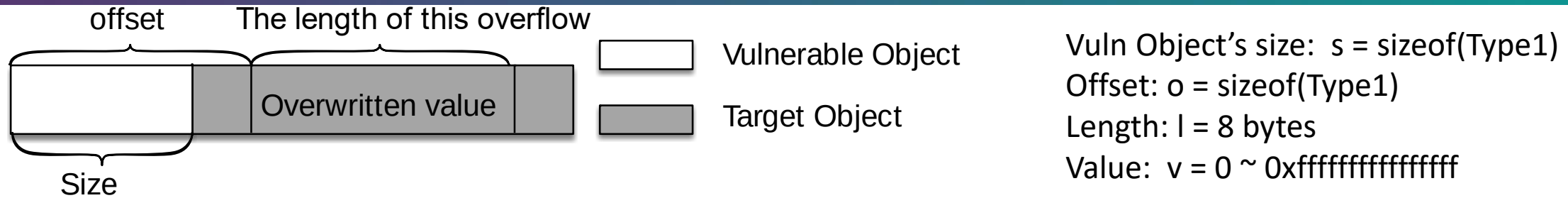
```
19. void sys_deref() { if (tgt->ptr) tgt->ptr(); } ← IP hijacking
```

Challenge 3: Exploitability Evaluation



- Heap layout arrangement: Heap Feng Shui
 - The behavior of Linux allocator is deterministic by default.

Challenge 3: Exploitability Evaluation

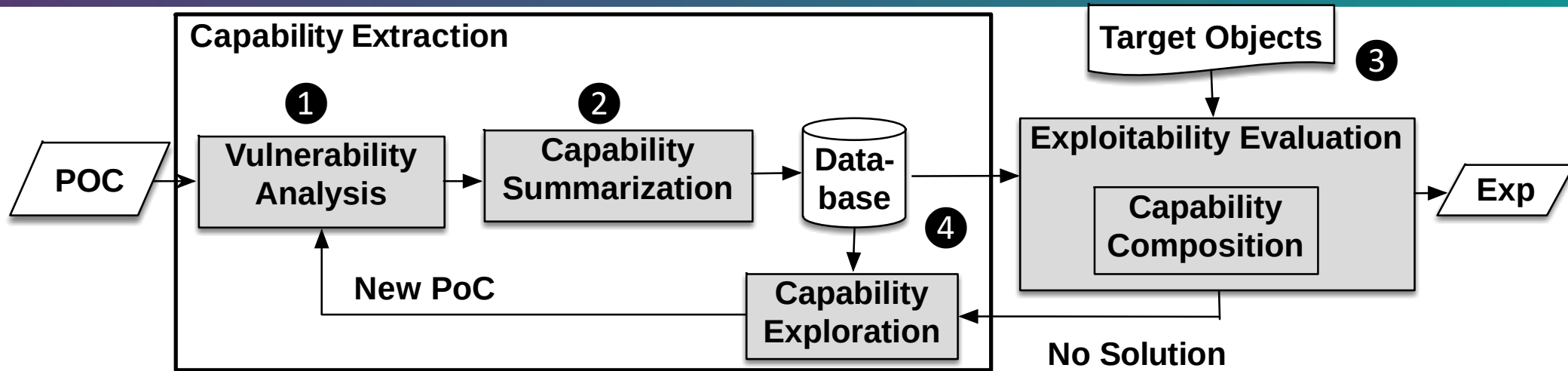


1. Heap layout arrangement: Heap Feng Shui
2. How to evaluate exploitability against different target objects?
3. How to efficiently search for suitable target objects among hundreds of candidates?

Scope and Assumption

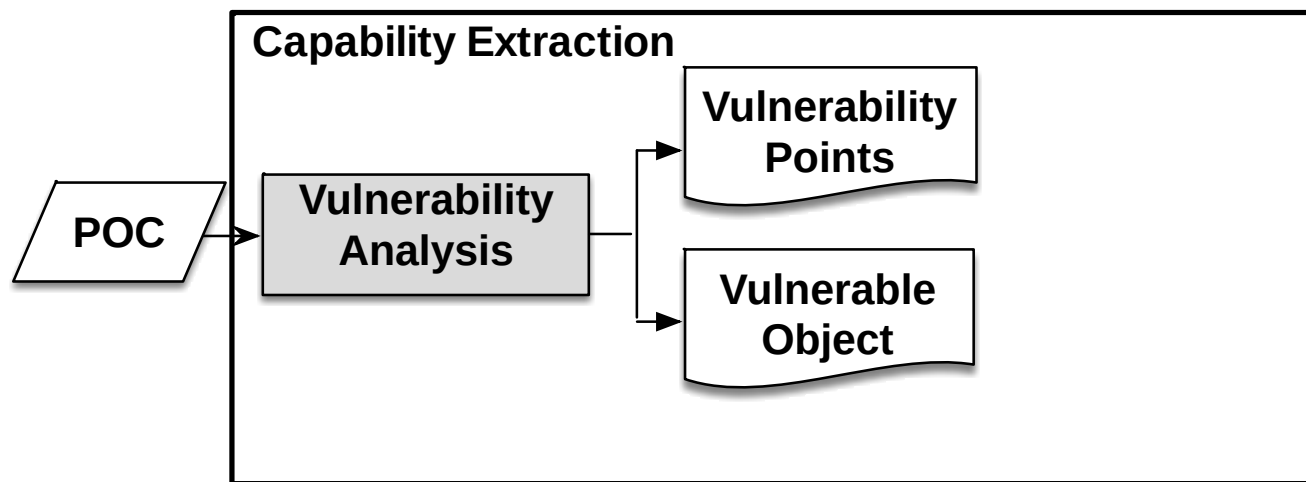
- Only generate exploit primitives to achieve IP hijacking
- Modern defenses are out of scope
 - Kernel Address Space Layout Randomization (KASLR)
 - Supervisor Mode Execution Prevention (SMEP)
 - Supervisor Mode Access Prevention (SMAP)
- Only encode some well-known heap Feng Shui strategies

Overview



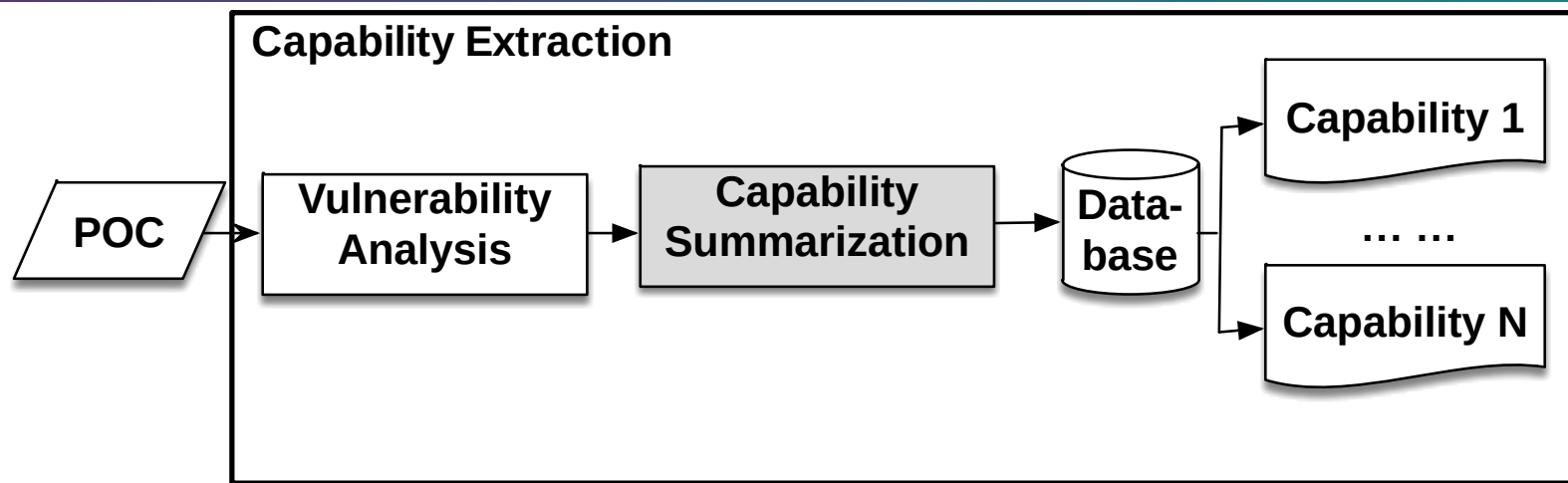
- Design principle
 - Separate capability summarization from exploitability evaluation

Overview



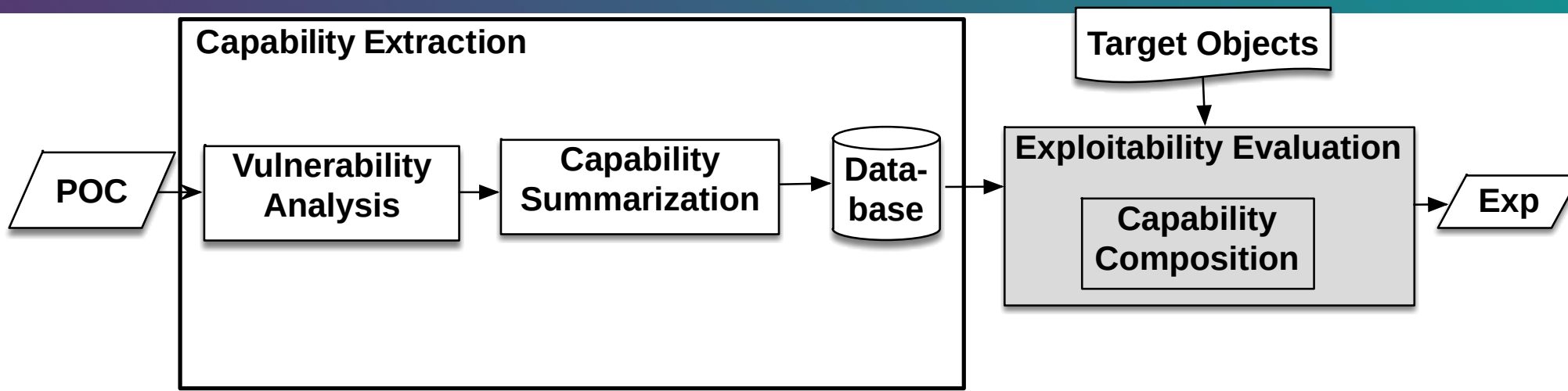
- Vulnerability Analysis
 - KASAN + Symbolic Execution
- Capability Summarization
- Exploitability Evaluation
- Capability Exploration

Overview



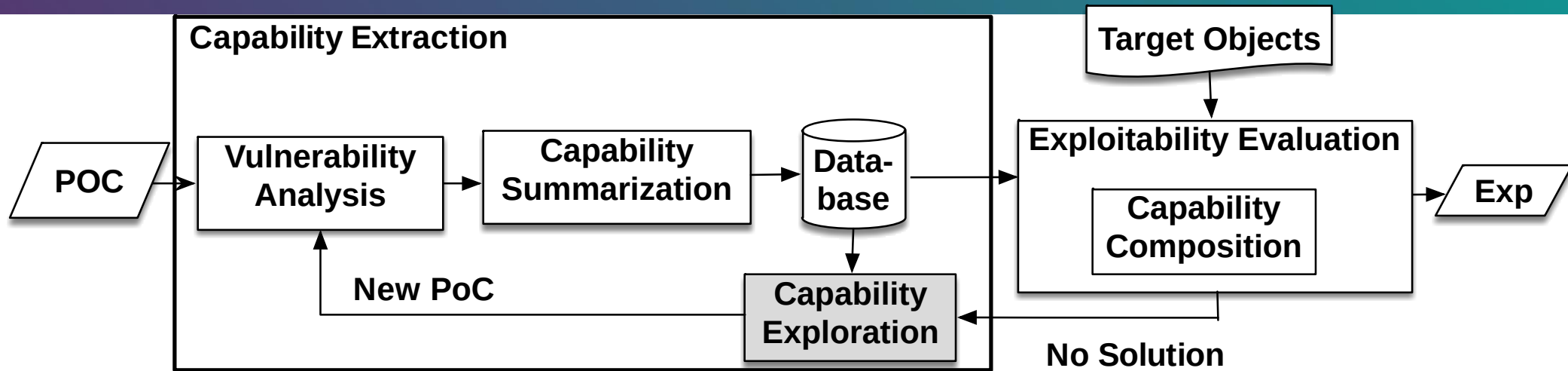
- Vulnerability Analysis
- Capability Summarization
 - Symbolic Execution
- Exploitability Evaluation
- Capability Exploration

Overview



- Vulnerability Analysis
- Capability Summarization
- Exploitability Evaluation
 - Constraint Solving
- Capability Exploration

Overview



- Vulnerability Analysis
- Capability Summarization
- Exploitability Evaluation
- Capability Exploration
 - Extend Syzkaller to support crash exploration mode

Symbolic Execution

- It is a static analysis technique that emulates the execution of a program.
- Each variable can be associated to a symbol
 - E.g., $\text{int } i \mapsto \alpha_i$
- Each symbol represents a set of values
 - E.g., $\alpha_i \in [0, 2^{32}-1]$
- Program statements generate formulas over symbols, e.g.,
 - E.g., $i * 2 + 5 \mapsto 2 * \alpha_i + 5$

Symbolic Execution

- Fork the execution on conditional branch
 - E.g., if ($i < 10$) { ... } else { ... }
 - Path 1: $\alpha_i < 10$
 - Path 2: $\alpha_i \geq 10$
- Since we have PoC, we can guide the symbolic execution to follow one path without forking.

Capability Summarization

- For each OOB write, we use a 3-tuple (***offset***, ***length***, ***value***) to present it.
 - *offset*, *length* and *value* are all symbolic expression.

```
8. void sys_socket() //sizeof(Type1) == sizeof(Type3)
9.   vul = kmalloc(sizeof(Type1))
10. void sys_accept()
11.   vul = (Type2*)vul; //type confusion
12.   vul->option = gsock.option; //Vulnerability Point
```

Assign a unique symbol
“ptr_vul” to the variable “vul”

ptr_vul + offsetof(Type2, option) = controllable_input

Base pointer

Offset

Length and Value

Capability Summarization

- For each OOB write, we use a 3-tuple (***offset***, ***length***, ***value***) to present it.
 - *offset*, *length* and *value* are all symbolic expression.

- ```
11. vul = (Type2*) vul; //type confusion
12. vul->option = gsock.option; //Vulnerability Point
```

**ptr\_vul + offsetof(Type2, option) = controllable\_input**

Base pointer

Offset

Length and Value

- For a single path, we consider a set of OOB write summarization.

# Capability Summarization

- **Path constraints** and the **size of the vulnerable object** that are coupled with OOB writes should be included.

```
13. void sys_setsockopt(val) //not invoked in given PoC
14. if (val == -1) return;
15. gsock.option = val;
```



# Capability Summarization

```
6. Type2 gsock = { ..., .option = 0x08080000000000, };
7. Type1 * vul = NULL; Type3 * tgt = NULL;
8. void sys_socket() //sizeof(Type1) == sizeof(Type3)
9. vul = kmalloc(sizeof(Type1))

10. void sys_accept()
11. vul = (Type2*)vul; //type confusion
12. vul->option = gsock.option; //Vulnerability Point

13. void sys_setsockopt(val) //not invoked in given PoC
14. if (val == -1) return;
15. gsock.option = val;
```

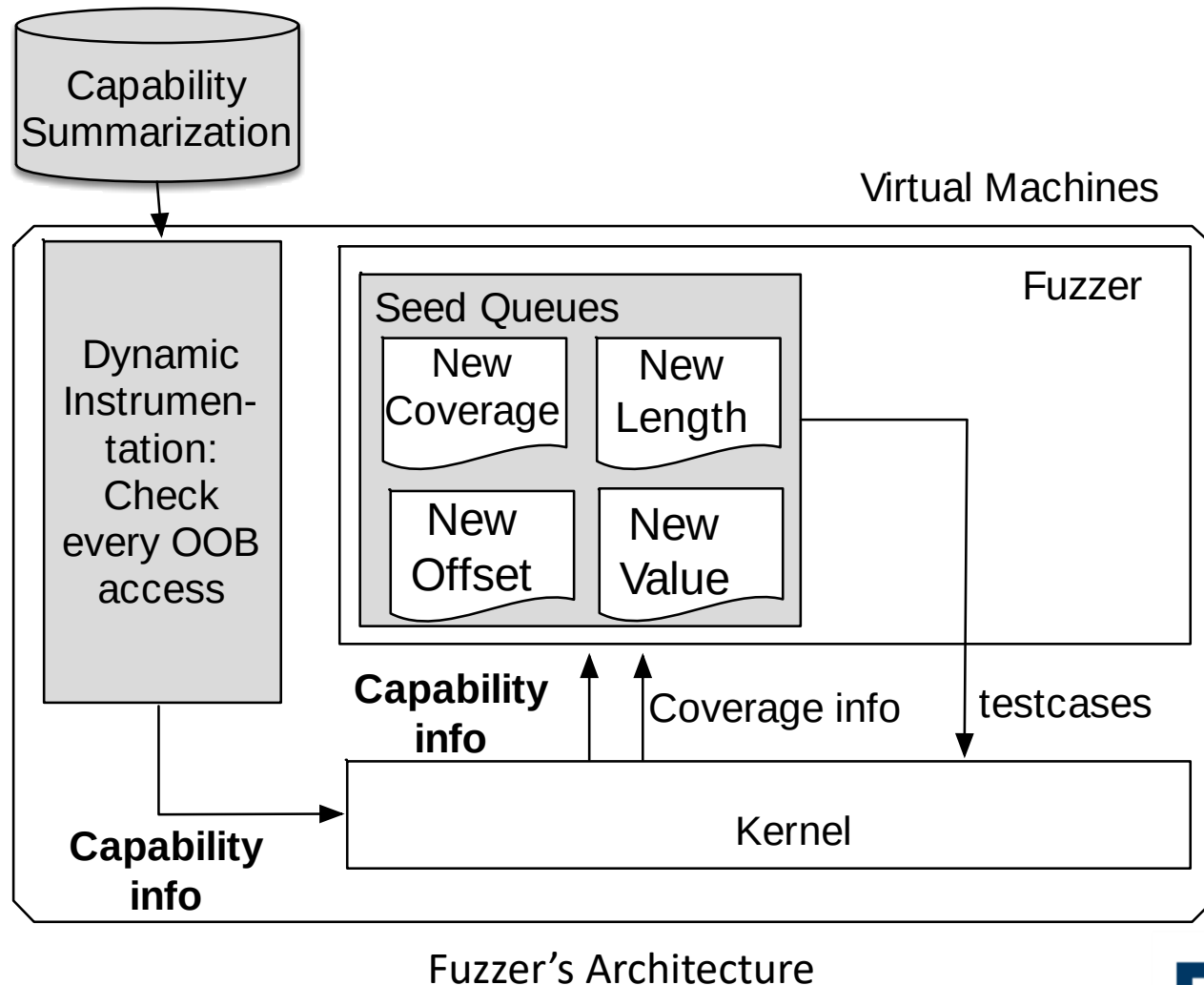
```
Ccomp = {
 sizeof(Type1),
 { val != -1 },
 { 12: (offsetof(Type2, option), 8, val) }
}
```

## Capability Comparison

$$C_{\text{orig}} \preceq C_{\text{comp}}$$

# Capability Exploration: Capability-Guided Fuzzing

- Existing coverage-guided fuzzers are insensitive to the capability.
- We use dynamic instrumentation to hook all the vulnerability points to collect information (e.g., offset, length, value) as the feedback.



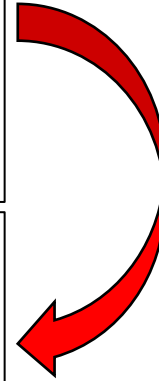
# Exploitability Evaluation: Target Objects

1. Function Pointer
2. Data Pointer
3. Non-Pointer:
  1. Uid
  2. Reference Counter

```
struct Type3 { int (*ptr)(); ...; };
```

```
16. void sys_create_tgt()
17. tgt = kmalloc(sizeof(Type3));
18. tgt->ptr = NULL;
19. void sys_deref() { if (tgt->ptr) tgt->ptr(); }
```

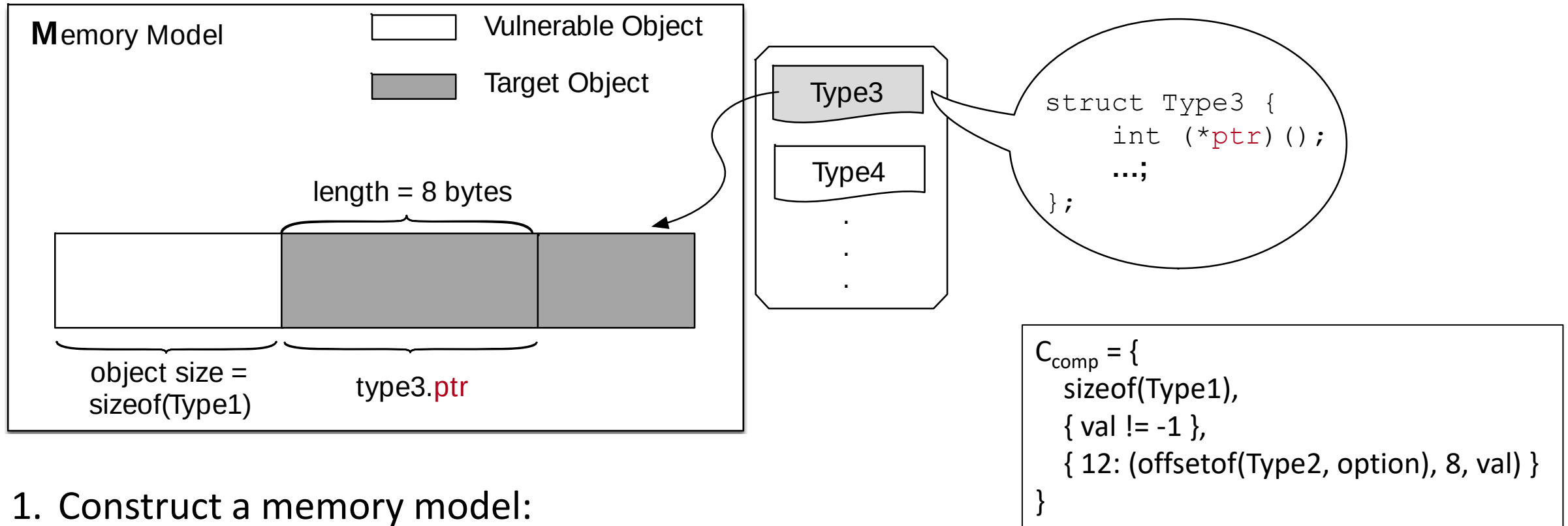
```
{
 "Type3": {
 "type": "function pointer",
 "offset": 0, // the offset to the function pointer
 "size": 192, // size of this object
 "payload": 0xdeadbeef,
 "allocate": // How to allocate this object
 "sys_create_tgt()",
 "deref": // How to trigger the dereference of the
 "sys_deref();",
 }
}
```



# Collecting Target Objects

- Manually collect promising target objects used in the wild
- Related work on automating the process:
  - A Systematic Study of Elastic Objects in Kernel Exploitation (CCS' 20)
  - SLAKE: Facilitating Slab Manipulation for Exploiting Vulnerabilities in the Linux Kernel (CCS' 19)

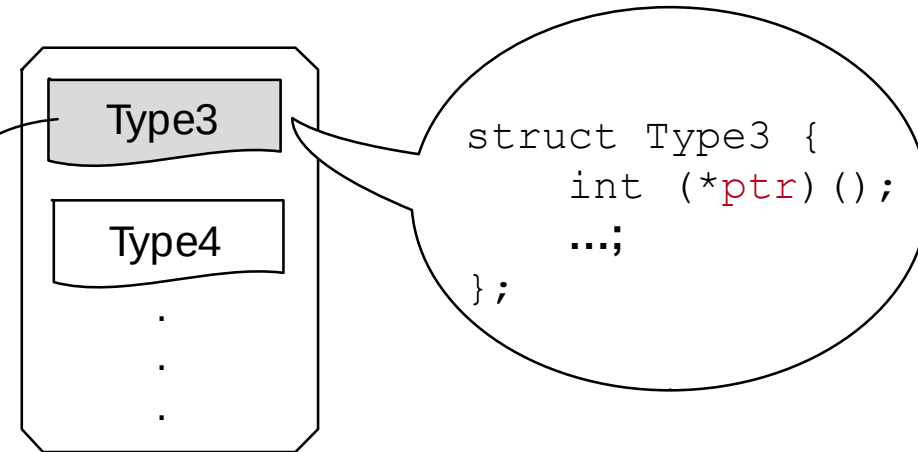
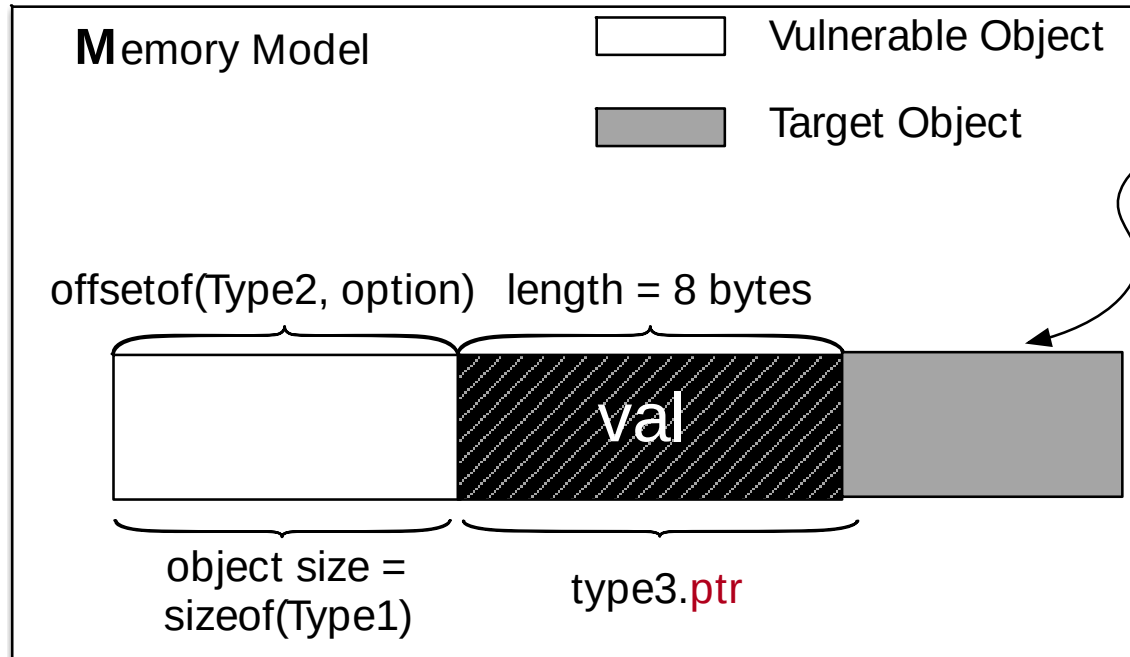
# Exploitability Evaluation



1. Construct a memory model:

$M[0 : \text{sizeof}(\text{Type1}) + \text{sizeof}(\text{Type3})]$  with concrete values

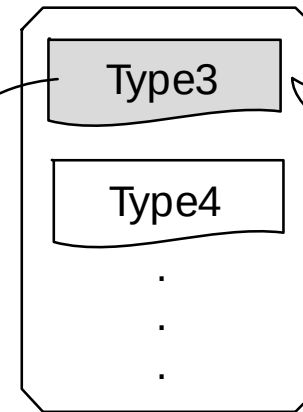
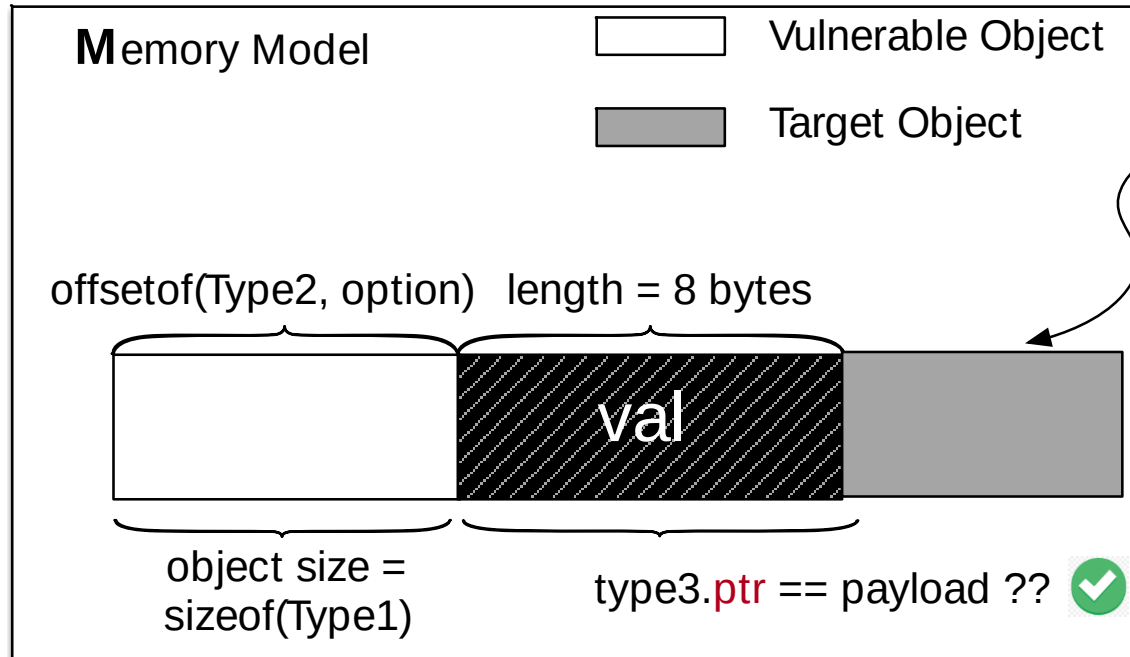
# Exploitability Evaluation



```
C_comp = {
 sizeof(Type1),
 { val != -1 },
 { 12: (offsetof(Type2, option), 8, val) }
}
```

1. Construct a memory model
2. Update the memory model with OOB writes  
 $M[\text{off} : \text{off} + 8] = \text{val}$

# Exploitability Evaluation

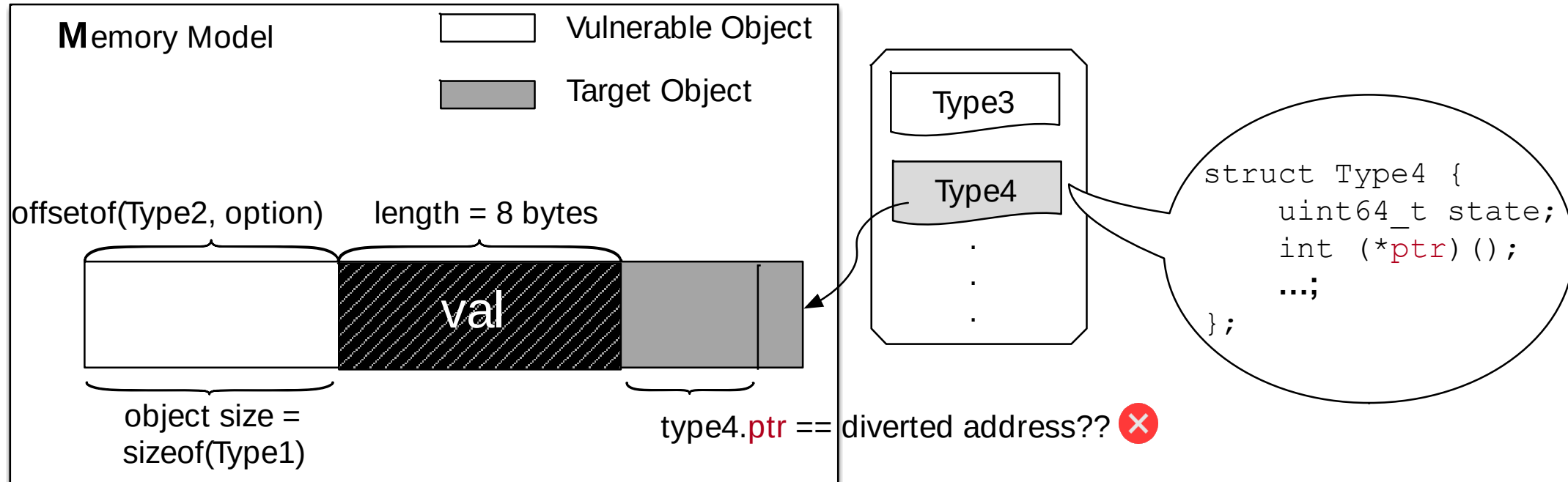


```
struct Type3 {
 int (*ptr)();
 ...;
};
```

```
C_comp = {
 sizeof(Type1),
 { val != -1 },
 { 12: (offsetof(Type2, option), 8, val) }
}
```

1. Construct a memory model
2. Update the memory model with OOB writes  
 $M[\text{off} : \text{off} + 8] = \text{val}$
3. Query SMT solver with respect to path constraints  
 $\text{typ3.ptr} == M[\text{off} : \text{off} + 8] == \text{0xdeadbeef} ??$

# Exploitability Evaluation



1. Construct a memory model
2. Update the memory model with OOB writes  
 $M[\text{off} : \text{off} + 8] = \text{val}$
3. Query SMT solver with respect to path constraints  
 $\text{typ4.ptr} == M[\text{off} + 8 : \text{off} + 16] == \text{0xdeadbeef}??$

# Walk-through

```
void example1(size)
 vul = kmalloc(size);
 vul[size] = '\0';
```

CVE-2016-6187

## PoC

1. `r0 = openat$apparmor_task_current(0xffffffffffffffff9c, &(0x7f0000000700)='/proc/self/attr/current\x00', 0x2, 0x0)`
2. `write(r0, &(0x7f0000000800)='11111111... 11111111', 0x100)`

# Walk-through

## PoC

```
1. r0 = openat$apparmor_task_current(0xffffffffffffffff9c,
 &(0x7f0000000700)='/proc/self/attr/current\x00', 0x2, 0x0)
2. write(r0, &(0x7f0000000800)='11111111... ..11111111', 0x100)
```

// autogenerated by KOOBE

```
2. syscall(__NR_mmap, 0x20000000, 0x1000000, 3, 0x32, -1, 0);
3. uint64_t size = 0x100;
4. memcpy(0x20000700, "/proc/self/attr/current\000", 24);
5. long res = syscall(__NR_openat, 0xffffffffffffffff9c,
6. 0x20000700, 2, 0);
8. memcpy((void*)0x20000800, "11111111... ..111111111111" 256);
15. s2e_make_concolic((void*)0x20000800, 256, "ptr_0x20000800");
16. s2e_make_concolic(&size, 8, "size");
17. syscall(__NR_write, res, 0x20000800, size);
```



**Selectively make  
arguments symbolic  
according to their  
type declared in  
Syzkaller**

# Walk-through

- Vulnerability Analysis

```
{
 "vuln_obj": {
 "size": 256, // Concrete value of the size
 "callsite": 0xfffffffff811f18d0 // The address of the function call allocating
the object
 },
 "OOB Writes": [
 {
 // call chain to the KASAN report function
 "backtrace": [0xfffffffff814b56a6, 0xfffffffff81477763],
 "length": 1
 }
]
}
```

# Walk-through

- Capability Summarization

```
void example1(size)
 vul = kmalloc(size);
 vul[size] = '\0';
```

← Hook the instruction to extract symbolic expression

CVE-2016-6187

```
Corig = {
 size,
 {},
 {0xffffffff814b56ad: (size, 1, 0x0) }
}
```

**size** is an unconstrained symbol.  
The size of the vulnerable object is  
the same as the offset of the OOB  
write.

# Walk-through

## • Exploitability Evaluation

```
"key": {
 "type": "reference counter",
 "offset": 0, // The offset to the critical field
 "size": 192,
 // The new value of the critical field
 "payload": "\x00\x00\x00\x00",
 "original value": "\x01\x00\x00\x00"
 // How to allocate this object
 "allocate":,
 // How to exploit it after we modify the critical field
 "dereference":,
}
```

```
"target": "key",
"size": 192, // The required size for the vulnerable object
"arguments": {
 "ptr_0x20000800": [49, 49, 49, 49],
 "size": [192, 0, 0, 0, 0, 0, 0, 0]
}
... ..
```



Produce a solution

# Walk-through

- Exploit Synthesis
  - Concretize the arguments
  - Add instrumentation to perform heap Feng Shui

```
1. int main(void) {
2. syscall(__NR_mmap, 0x20000000, 0x1000000, 3, 0x32, -1, 0);
3. uint64_t size = 192;
4. memcpy(0x20000700, "/proc/self/attr/current\000", 24);
5. r[0] = syscall(__NR_openat, 0xffffffff9c, 0x20000700, 2, 0);
6. do_fengshui();
7. do_fengshui_tgt();
8. do_alloc_target();
9. do_fengshui_vuln();
10. memcpy(0x20000800, "\x31\x31 \x31\x31", 192);
11. syscall(__NR_write, r[0], 0x20000800, size);
12. do_fengshui_trigger();
13. do_trigger();
14. return 0;
```

# Evaluation

- Dataset and Setup
  - 7 from CVE database
  - 10 from syzbot (a fuzzing platform based on Syzkaller)

# Evaluation

- Dataset and Setup
  - 7 from CVE database
  - 10 from syzbot (a fuzzing platform based on Syzkaller)

- Results

| CVE-ID           | Race Condition | #public EXP | #generated EXP* |
|------------------|----------------|-------------|-----------------|
| CVE-2016-6187    | No             | 1           | 2               |
| CVE-2016-6516    | Yes            | 0           | 0               |
| CVE-2017-7184    | No             | 1           | 3               |
| CVE-2017-7308    | No             | 1           | 2               |
| CVE-2017-7533    | Yes            | 0           | 1               |
| CVE-2017-1000112 | No             | 1           | 2               |
| CVE-2018-5703    | No             | 0           | 1               |
| Overall          |                | 4           | 11              |

\*: We count the number of distinct exploits based on the target object we exploit.

# Evaluation

| Commit Hash                              | #public EXP | #generated EXP |
|------------------------------------------|-------------|----------------|
| 813961de3ee6474dd5703e883471fd941d6c8f69 | 1           | 2              |
| 35f7d5225ffcbf1b759f641aec1735e3a89b1914 | 0           | 2              |
| bbeb6e4323dad9b5e0ee9f60c223dd532e2403b1 | 0           | 2              |
| eb73190f4fbeedf762394e92d6a4ec9ace684c88 | 0           | 1              |
| 4576cd469d980317c4edd9173f8b694aa71ea3a3 | 0           | 1              |
| 17cfe79a65f98abe535261856c5aef14f306dff7 | 0           | 0              |
| 9fa68f620041be04720d0cbfb1bd3ddfc6310b24 | 0           | 0              |
| 3619dec5103dd999a777e3e4ea08c8f40a6ddc57 | 0           | 0              |
| 70303420b5721c38998cf987e6b7d30cc62d4ff1 | 0           | 0              |
| bb29648102335586e9a66289a1d98a0cb392b6e5 | 0           | 0              |
| Overall                                  | 1           | 8              |

# Time Cost

| CVE or Commit        | Symbolic Execution | Solving | Fuzzing | Time to Patch       |
|----------------------|--------------------|---------|---------|---------------------|
| CVE-2016-6187        | 38s                | 1s      | NA      | NA                  |
| CVE-2017-7184        | 27s                | 45s     | 23m     | 14 days (estimated) |
| CVE-2017-7308        | 48s                | 4s      | NA      | NA                  |
| CVE-2017-7533        | 160s               | 164s    | NA      | NA                  |
| CVE-2017-1000112     | 36s                | 132s    | NA      | NA                  |
| CVE-2018-5703        | 85s                | 41s     | 194m    | 114 days            |
| 813961de3ee6474dd570 | 34s                | 5s      | NA      | 12 days             |
| 35f7d5225ffcbf1b759f | 34s                | 18s     | 8m      | 50 days             |
| bbeb6e4323dad9b5e0ee | 48s                | 26s     | 23m     | 11 days             |
| eb73190f4fbeedf76239 | 54s                | 104s    | NA      | 38 days             |
| 4576cd469d980317c4ed | 57s                | 7s      | NA      | 31 days             |

Note: We only apply fuzzing when necessary, i.e., our system is unable to find a suitable target object given the capability in the original PoC.

# Takeaways

- One vulnerability can manifest different security impacts
- Different OOB vulnerability instances exhibit a wide range of capabilities
- Kernel exploits are multi-interaction in nature which allows the exploit crafting process to be modular

# THANK YOU

## ANY QUESTION?

Contact: [wchen130@ucr.edu](mailto:wchen130@ucr.edu)

Paper: “KOOBE: Towards Facilitating Exploit Generation of Kernel Out-Of-Bounds Write Vulnerability” USENIX Security 2020

Code: <https://github.com/seclab-ucr/KOOBE>